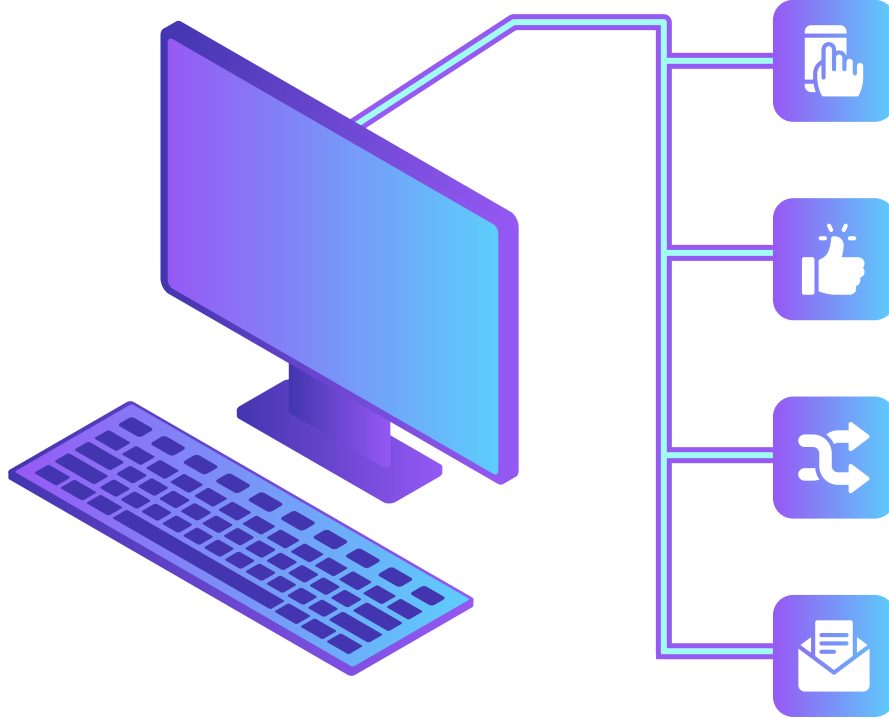


Shopping Lists on the Cloud



Diogo Sarmiento up202109663
Rodrigo Póvoa up202108890
Tomás Sarmiento up202108778

Index



Context

Description of the project and requirements

Technology

Technologies used for the project

CRDT

Add Wins Observed Remove Set

Project Architecture and Features

Which features support our project, such as replication...

Context

Local-First Shopping List App

- This project focuses on building a shopping list application with both local and cloud functionality.
- Users can create and share lists via unique IDs, enabling collaborative item management.
- Items can have flags or target quantities for tracking.
- The app uses AWORSET CRDT for conflict resolution and scalability
- The cloud architecture is designed to handle millions of users, leveraging sharded, independent list data for efficiency.



Technologies Used

Node.js

For the client and server applications

HTML/CSS

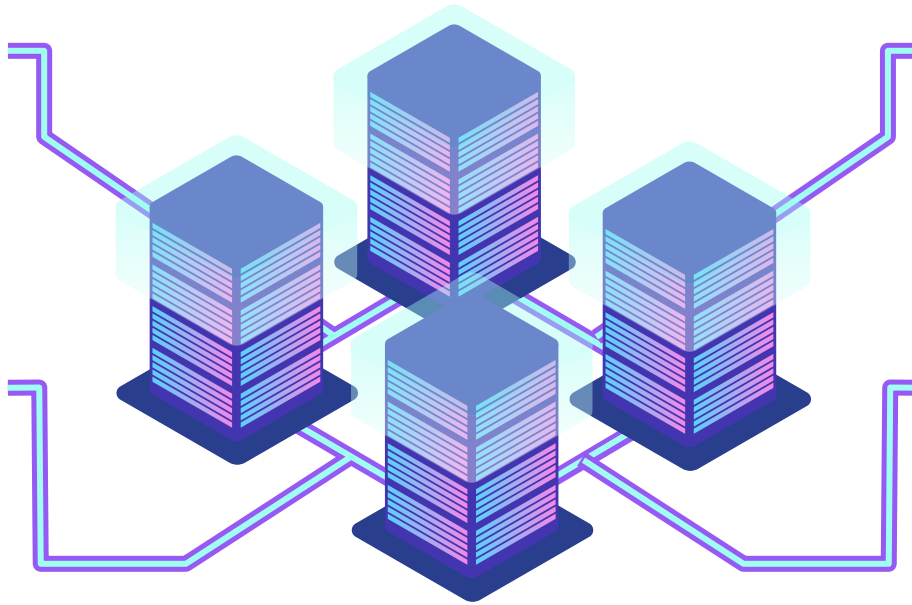
For the development of the web application

ZeroMQ

For high-level and performance asynchronous messaging

JSON

For database storage



CRDT implementation



AworSet

Aworset was used to add and remove the whole quantity of the item in the list. Was chosen because of its a OrMap style we need to implement because we have to track the values of differently named products and where the addition of a element wins over the removal in the resolution of conflicts.

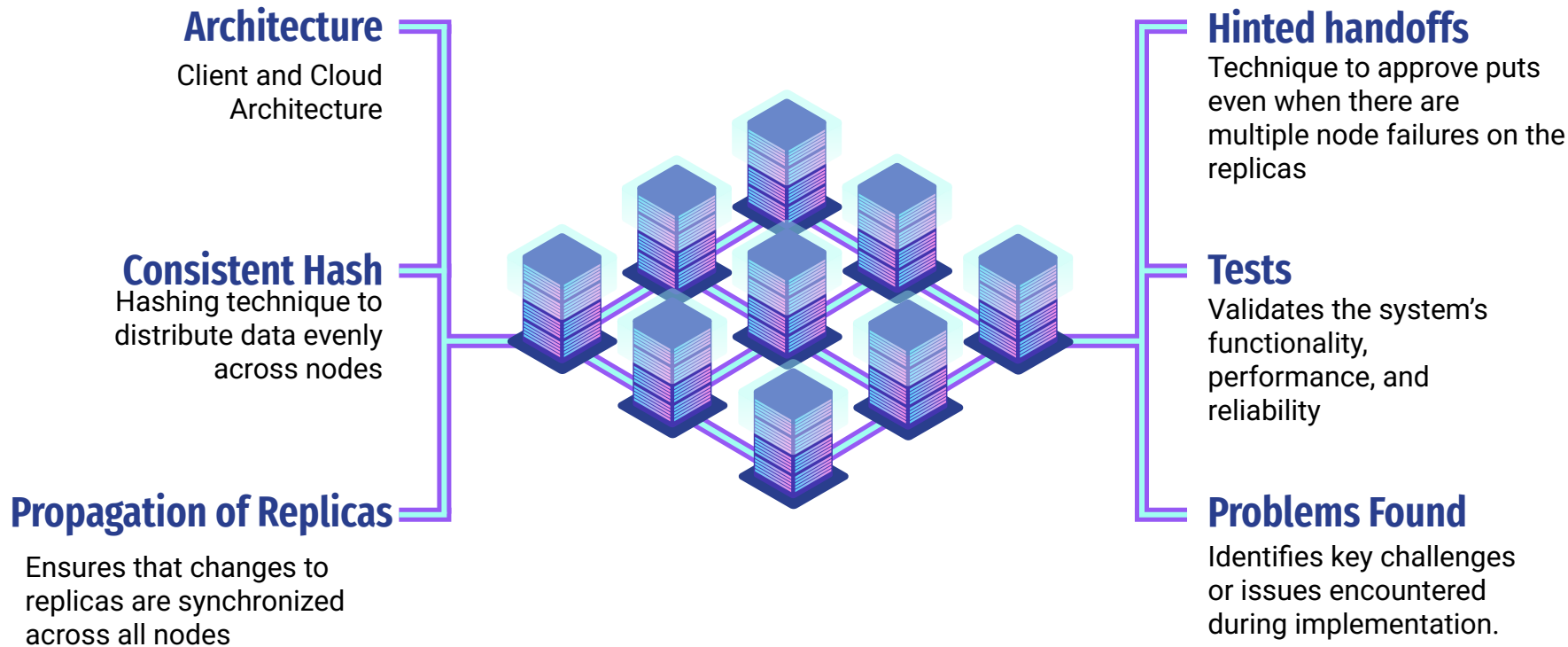


PNCounter

For the track of the individual product value was used PNCounter which is able to successfully merge changes of values across different clients.



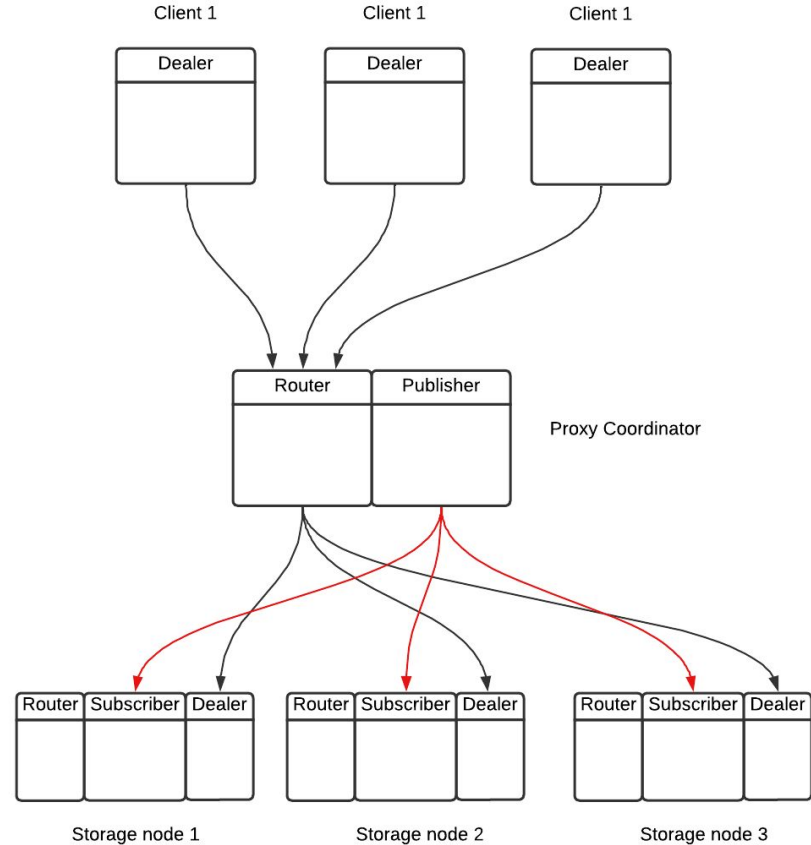
Project Architecture and Features



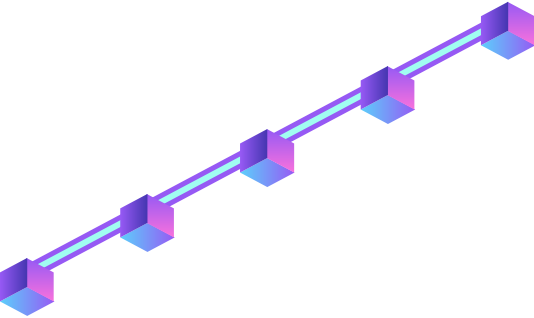
Architecture

Architecture

- **Client:** each client has a zmq dealer to create requests to the proxy/Coordinator
- **Proxy Coordinator:** each Proxy Coordinator after receiving a request from the router calculate the preference list and resend the request to the first node on the preference list.
- **StorageNodes:** each Storage node have a subscriber socket to receive updates on the ring, a dealer to register the node and receive requests from the router and a router to receive requests from other storages nodes. To propagate a put command each storage node creates a Request Socket and sends it to the router of the according router of a Storage Node.



Consistent Hashing



Consistent Hashing Overview

- **Purpose:** Efficiently distribute data across a cluster of nodes, minimizing reorganization when nodes are added or removed.
- **Hash Ring:** A circular structure where each node and data item is assigned a position based on a hash value.
- **Virtual Nodes:** Multiple positions in the hash ring for each physical node to ensure even data distribution.
- **Replication** : Each list is replicated in the top k vnodes on the preference list in the hash ring.

Node Management

- **Adding Nodes:** New nodes are assigned multiple positions (virtual nodes) in the hash ring, requiring minimal data redistribution.

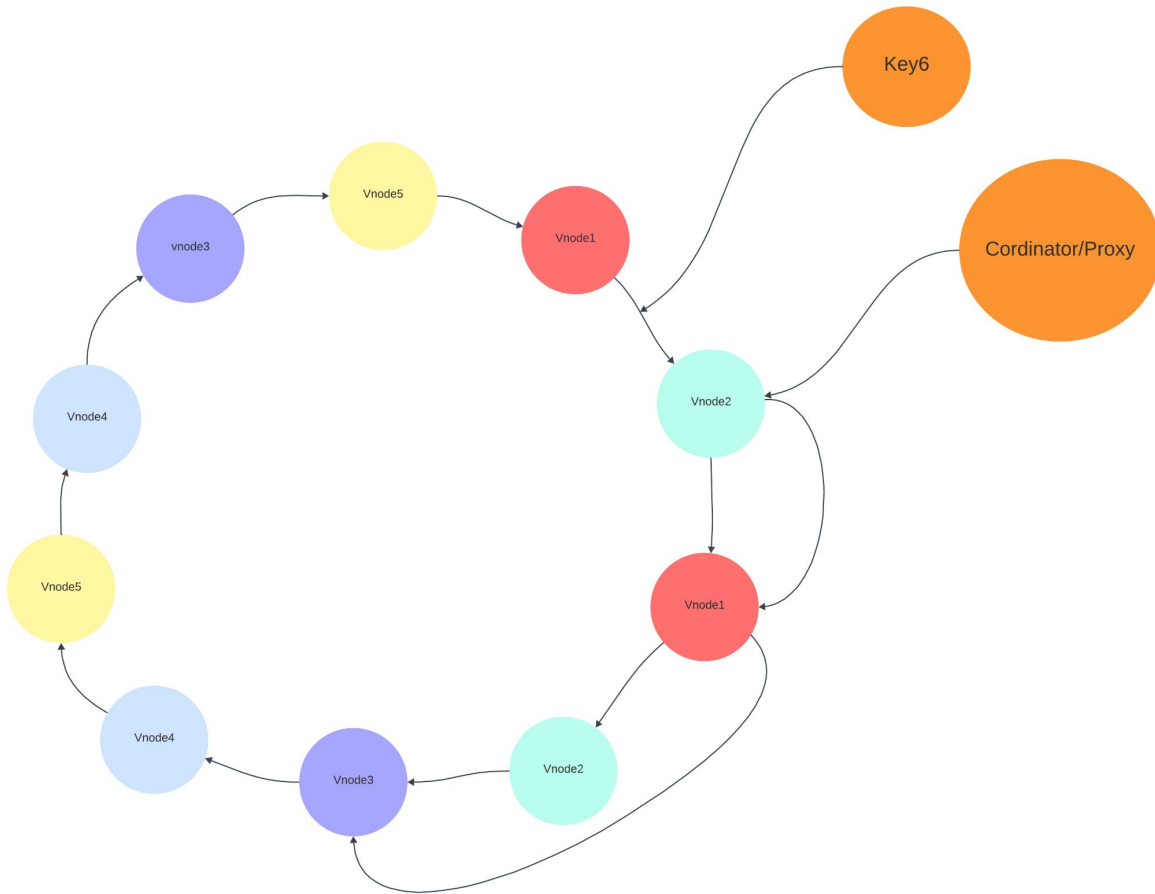
Data Distribution

- **Key Mapping:** Data items are assigned positions in the hash ring based on their hash values.
- **Node Assignment:** Data is stored in the first node that appears after its position in the hash ring.

Propagation of Replicas of Lists

Propagation of Replicas

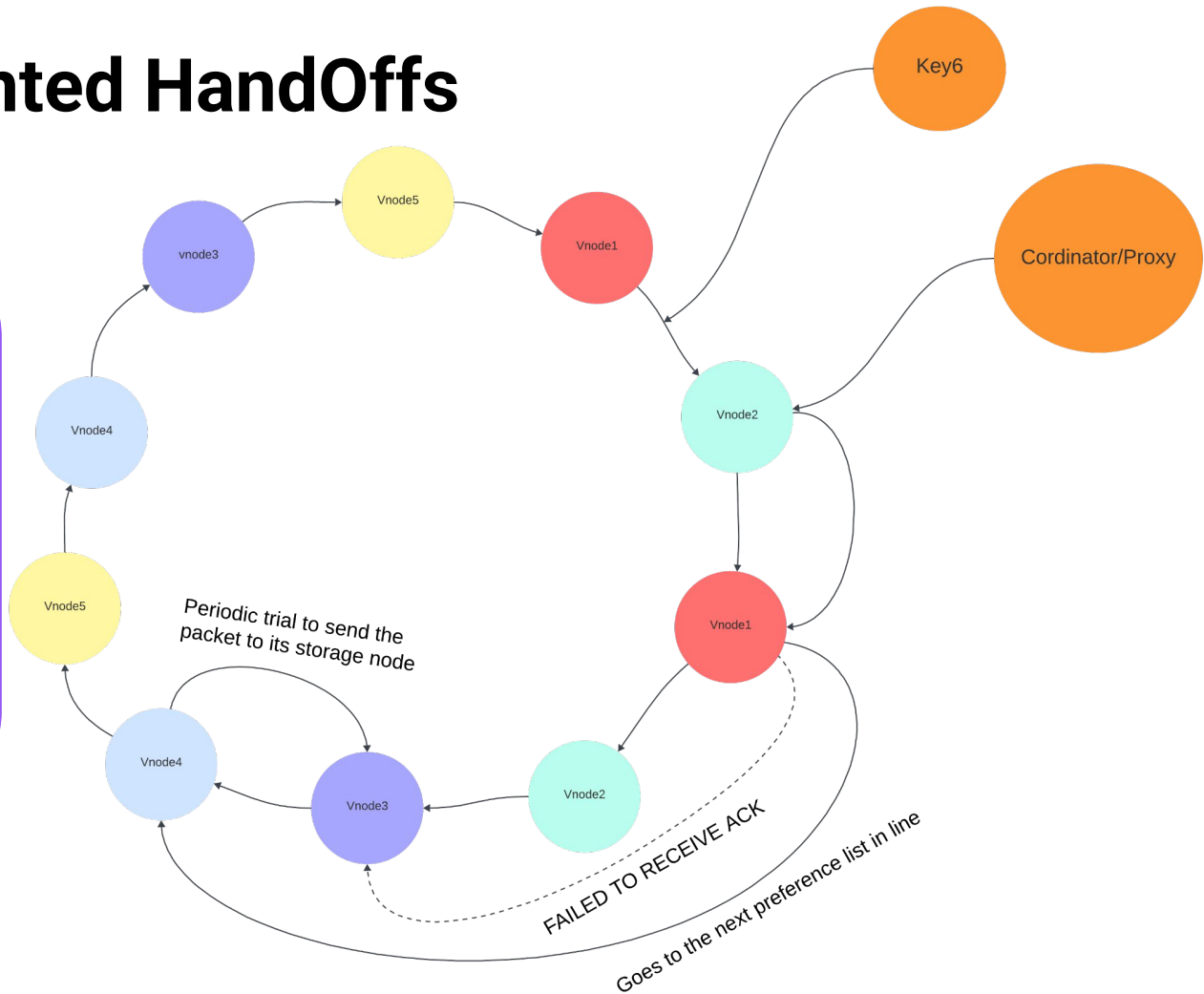
-Propagation: Each node receives a request from a router, if it is from the coordinator, or it receives a request from a request socket from one previous storage node on the preference list, the request is propagated by each storage node until it has the required approved replicas written. When it has the required replicas approved it backtracks and send an ok.



Hinted HandOffs

Sloppy quorum

Approving a request using the top k healthy nodes on the preference list, so that the requests can be approved even in a node failure, maintaining the replicas online. When the failed nodes comeback online using hinted handoffs we can send the list meant for that node later the quorum is write 3, read 1, replicas 3. We follow the following quorum. $R + W > N$



Validation Tests

Storage Failure

In this test, we tested the Put and Get request functions during a storage node failure, and recovery

Multiple storage failure

In this test, we tested the Put and Get request functions during multiple node failures and recovery



Coordinator failure and rapid restart

In this test, we tested the ability for the system to recover if the coordinator crashed and recovered rapidly

Client Connection

Client handles cases where server isn't online
Testing Set and Get function

Web User interface

get existing list

Check List

Create New Shopping List

Create List

Existing Shopping Lists

tomas

OpenDelete

tomas

Enter item name

1

Add Item

pineapple

-4+

Remove

apple

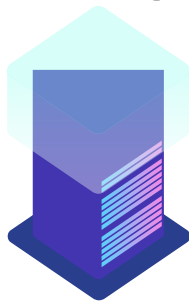
-6+

Remove

Sync List

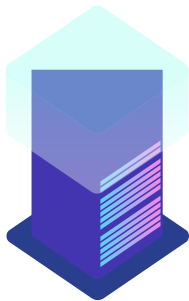
Challenges

1 Using ZMQ sockets, primarily Request sockets, can be challenging to test due to their extensive internal functionality.



2 Example 1: If we attempt to make the same connection multiple times without receiving a response, the connection simply fails. We tried to introduce noise during `.receive()` on the router, but the sockets weren't receiving requests. Even after adjusting the high-water mark, there was no noticeable change

3 When dealing with parallel tasks, it's crucial to use mutexes for everything that cannot operate in parallel, such as storage, `router.send`, and more.



4 After making the system as robust as it is, demonstrating each functionality in action becomes difficult. The console outputs an overwhelming amount of information, making it challenging to identify the most relevant events in the Storage Nodes.



5 Using Virtual Nodes Just Complicates everything, from rebalancing to dynamically adding nodes, there are times when we can't add vnodes of the same storage Node.